

WITH_Server

성능분석보고서

2026. 01. 22

* 개요

날짜: 2026. 01. 22 (목)

빌드 설정: Release / x64

테스트 환경

- CPU: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz
- 스레드 수: Network Thread 8, Logic Thread 4
- OS: Windows10 Home

테스트 시나리오

- WITH_StressTest 프로그램 사용 (Move only)
- 엔티티 수: 100
- 세션 수: 100

* 성능 분석

Network Thread

io_context::run이 79.22% /
GetQueuedCompletionStatus가 19.42%를 차지하지만
이는 Asio 내부에서 IOCP 완료 대기 위한 함수이므로
Bottle Neck으로 평가하기 어려움.

Logic Thread

Game::Update가 15.13%, 그 중에서도
OutputEventsystem이 15.11%를 차지하여 Bottle Neck으로 보임.

OutputEventSystem에서도 Connection::Send 함수가 12.95%를
차지하는 것으로 보아 OutputEventSystem에서 네트워크 IO작업을
많이 수행하는 것이 문제인 것으로 판단됨.

with_server.exelasio::detail::win_iocp_io_context::run	24,078	8	79.22%	0.03%
with_server.exelasio::detail::win_iocp_io_context::do_o	24,070	517	79.19%	1.70%
with_server.exelasio::detail::executor_op<asio::detail::	15,246	56	50.16%	0.18%
kernelbase.dll!GetQueuedCompletionStatus	5,903	66	19.42%	0.22%
with_server.exelasio::detail::win_iocp_socket_send_o	2,247	29	7.39%	0.10%
with_server.exe	5,021	0	16.52%	0.00%
with_server.exe!_scrt_common_main_seh	5,021	0	16.52%	0.00%
with_server.exe!main	5,021	0	16.52%	0.00%
with_server.exe!Framework::Start	5,021	103	16.52%	0.34%
with_server.exe!Game::Update	4,598	0	15.13%	0.00%
with_server.exe!OutputEventSystem::Execute	4,592	0	15.11%	0.00%
with_server.exe!OutputEventSystem::ProcessMove	4,584	2	15.08%	0.01%
with_server.exe!ServerConnectionListener::Br	4,557	186	14.99%	0.61%
with_server.exe!Connection::Send	3,937	183	12.95%	0.60%
ntoskrnl.exe	707	0	2.33%	0.00%

* 최적화 방법 (개요)

Send호출 Batching

Concurrency Visualizer 분석 결과, Game::Update 내부에서 OutputEvent 처리 과정 중 다수의 Connection::Send 호출이 로직 스레드의 주요 병목으로 확인되었다.

기존의 구조는 Packet 단위로 즉시 Send를 호출하는 방식이었으며, 이로 인해 동일 Tick 내에서 반복적인 Send 호출이 발생하고 있었다.

해당 최적화는 Send 호출 횟수를 줄이는 것을 목표로, Tick 단위로 전송 요청을 수집한 뒤 일괄 Send하는 방식을 적용하였다.

이는 Tick 단위의 지연이 발생할 수 있는 단점이 있지만, 고정 Tick으로 로직이 수행되는 현재 서버 구조상 허용 가능하다고 판단하였다.

```
Update: avg=14.784ms min=7.839ms max=33.308ms
Update: avg=12.248ms min=7.703ms max=33.483ms
Update: avg=11.796ms min=6.788ms max=126.138ms
Update: avg=10.866ms min=7.549ms max=33.383ms
Update: avg=11.627ms min=7.768ms max=28.315ms
Update: avg=12.024ms min=7.092ms max=43.714ms
Update: avg=15.049ms min=7.460ms max=34.955ms
Update: avg=14.171ms min=7.122ms max=31.427ms
Update: avg=14.191ms min=7.721ms max=30.504ms
Update: avg=12.872ms min=6.765ms max=33.019ms
Update: avg=14.813ms min=8.205ms max=35.524ms
Update: avg=18.355ms min=7.794ms max=33.472ms
Update: avg=15.203ms min=7.462ms max=31.836ms
Update: avg=13.673ms min=7.644ms max=37.102ms
Update: avg=14.457ms min=6.870ms max=30.186ms
Update: avg=12.263ms min=6.794ms max=31.092ms
Update: avg=14.864ms min=7.369ms max=35.245ms
Update: avg=16.484ms min=6.967ms max=32.761ms
Update: avg=12.340ms min=7.168ms max=29.166ms
Update: avg=13.832ms min=6.836ms max=29.164ms
Update: avg=16.264ms min=8.300ms max=32.311ms
Update: avg=18.110ms min=8.326ms max=32.400ms
Update: avg=17.943ms min=7.312ms max=32.041ms
Update: avg=18.204ms min=7.420ms max=31.058ms
Update: avg=11.828ms min=7.452ms max=26.284ms
```

OutputEventSystem::Execute
60Tick 당 평균, 최소, 최대 처리 시간

* 최적화 방법 (구현)

1) 고정 크기 SendBuffer의 문제

현재 모든 Send 데이터는 SendBuffer의 형태로 관리되고 있다.

하지만 SendBuffer가 고정 크기로 구현되어있어 오버플로우의 위험이 있고, 배칭을 위해 SendBuffer에 계속해서 Packet Data를 추가하면 그 위험성이 더 심각하다 판단하였다.

이를 해결하기 위해 우선 SendBuffer를 가변 크기로 수정하고 이에 맞춰 SendBufferPool을 개선하였다.

* 최적화 방법 (구현)

1) 고정 크기 SendBuffer의 문제

현재 모든 Send 데이터는 SendBuffer의 형태로 관리되고 있다.

하지만 SendBuffer가 고정 크기로 구현되어있어 오버플로우의 위험이 있고, 배칭을 위해 SendBuffer에 계속해서 Packet Data를 추가하면 그 위험성이 더 심각하다 판단하였다.

이를 해결하기 위해 우선 SendBuffer를 가변 크기로 수정하고 이에 맞춰 SendBufferPool을 개선하였다.

* 최적화 방법 (구현)

2) unordered_map 기반 배치 구현

대충 unordered_map으로 먼저 구현함

매번 unordered_map 탐색하는 것을 해결할 수 있을거 같음

그래서 vector 기반(연속 배열)로도 구현해보기로 함

```
Update: avg=0.429ms min=0.289ms max=0.690ms
Update: avg=0.475ms min=0.343ms max=0.611ms
Update: avg=0.478ms min=0.280ms max=0.670ms
Update: avg=0.471ms min=0.314ms max=0.620ms
Update: avg=0.486ms min=0.312ms max=0.792ms
Update: avg=0.467ms min=0.262ms max=0.703ms
Update: avg=0.452ms min=0.294ms max=0.641ms
Update: avg=0.450ms min=0.273ms max=0.574ms
Update: avg=0.452ms min=0.275ms max=0.657ms
Update: avg=0.457ms min=0.272ms max=0.589ms
Update: avg=0.459ms min=0.286ms max=0.632ms
Update: avg=0.465ms min=0.288ms max=0.688ms
Update: avg=0.460ms min=0.288ms max=0.766ms
Update: avg=0.451ms min=0.295ms max=0.618ms
Update: avg=0.443ms min=0.304ms max=0.954ms
Update: avg=0.443ms min=0.282ms max=0.619ms
Update: avg=0.474ms min=0.271ms max=0.720ms
Update: avg=0.463ms min=0.301ms max=0.638ms
Update: avg=0.482ms min=0.280ms max=0.693ms
Update: avg=0.444ms min=0.289ms max=0.645ms
Update: avg=0.453ms min=0.280ms max=0.586ms
Update: avg=0.463ms min=0.284ms max=0.598ms
Update: avg=0.489ms min=0.284ms max=0.710ms
Update: avg=0.484ms min=0.324ms max=0.653ms
Update: avg=0.507ms min=0.366ms max=0.700ms
```

OutputEventSystem::Execute
60Tick 당 평균, 최소, 최대 처리 시간

* 최적화 방법 (구현)

3) vector 기반 배칭 구현

아직 엔티티가 100개 밖에 없어서 그런지 별로 차이가 안남

일단 현재 상태에서는 보류하고 나중에 더 부하가 커지면 다시 테스트해보기로 결정함

```
Update: avg=0.440ms min=0.259ms max=0.568ms
Update: avg=0.429ms min=0.306ms max=0.706ms
Update: avg=0.449ms min=0.294ms max=0.586ms
Update: avg=0.438ms min=0.260ms max=0.595ms
Update: avg=0.449ms min=0.295ms max=0.624ms
Update: avg=0.448ms min=0.263ms max=0.605ms
Update: avg=0.437ms min=0.272ms max=0.928ms
Update: avg=0.411ms min=0.232ms max=0.599ms
Update: avg=0.439ms min=0.257ms max=0.596ms
Update: avg=0.427ms min=0.256ms max=0.626ms
Update: avg=0.450ms min=0.239ms max=0.638ms
Update: avg=0.432ms min=0.293ms max=0.580ms
Update: avg=0.455ms min=0.274ms max=0.895ms
Update: avg=0.422ms min=0.275ms max=0.576ms
Update: avg=0.463ms min=0.279ms max=0.638ms
Update: avg=0.495ms min=0.252ms max=1.027ms
Update: avg=0.477ms min=0.272ms max=0.751ms
Update: avg=0.449ms min=0.300ms max=0.757ms
Update: avg=0.454ms min=0.308ms max=0.623ms
Update: avg=0.484ms min=0.285ms max=0.651ms
Update: avg=0.468ms min=0.246ms max=0.695ms
```

OutputEventSystem::Execute
60Tick 당 평균, 최소, 최대 처리 시간

* 최적화 전후 성능 비교

- 동일 시나리오 적용
- 1000 Tick 기준

항목	Before	After (vector 구현)	Diff
평균	14.319ms	0.453ms	-96.8%
최대	36.315ms	1.164ms	-96.7%
최소	7.429ms	0.003ms	-99.9%